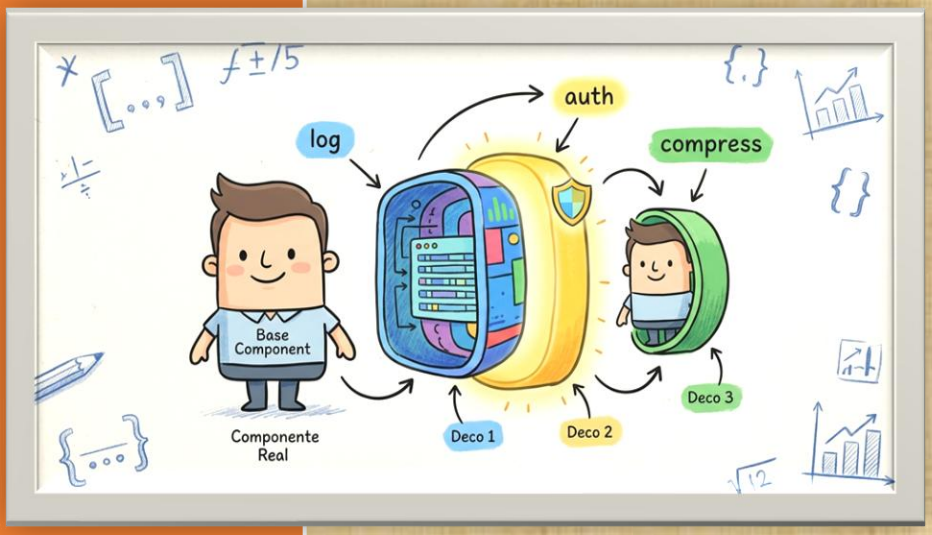
Trabajo Práctico Final
Patrones de Diseño

Estudio un patrón y lo muestro:



- Nombres del alumno: Benjamin Ezequiel
- Apellido del Alumno: Duarte Maydana
- Instituto: Instituto Superior de Formación Docente y Técnica Nº 59
- Materia: Algoritmos y Estructuras de Datos.
- Profesor: Santos Silva Martin Fidel
- Fecha: 18/10/2026
- Fecha de entrega: 4/noviembre/2026

Contenido:

- **Introducción: ¿Qué es un patrón de diseño y cuál veremos en este documento?** 3

1) El problema que resuelve: 4

- **El problema:** 4
- **Dilema Técnico:** 4
- **Los tres dolores puntuales a la hora de desarrollar:** 4
- **EL problema: "Explosión de clases y Rigidez Estética** 5

Código sin el patrón: El enfoque por herencia: 5

¿Dónde le duele a este diseño? 7

Siendo más técnico: 8

2) ¿Por qué usar este patrón y no otro? 9

- **Por qué la solución simple deja de ser útil**: 9

3). La Estructura del Patrón 10

- **Descripción de Roles y Responsabilidades:** 11

4). Ejemplo concreto en java 13

- **Diagrama U.L.M del ejemplo**: 15
- **Código de ejemplo en java : (personaje RPG)** 17

5). Uso Real y Cuándo NO Usarlo 26

- **Un caso concreto donde el patrón sobra o empeora el diseño:** 26
- **La solución simplificada (Cómo debió resolverse)** 27

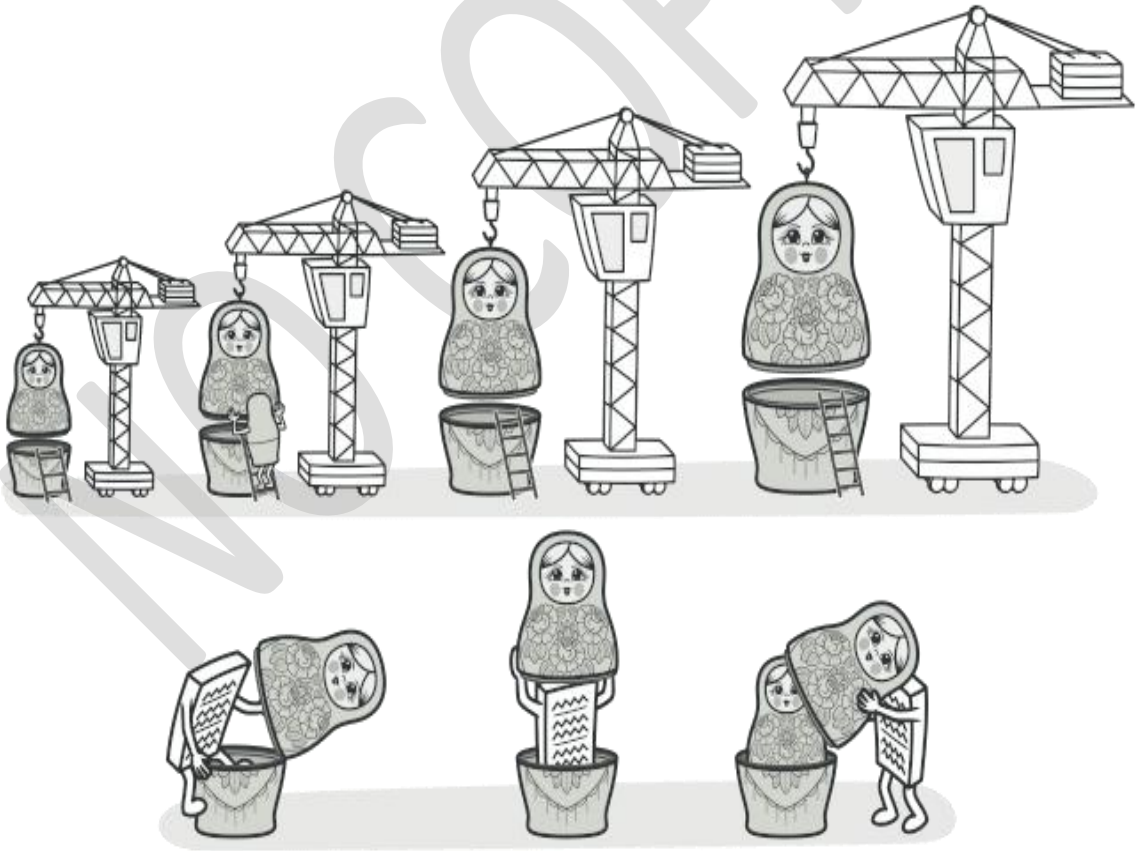
6). Fuentes: 28

NO COPIAR

• Introducción: ¿Qué es un patrón de diseño y cuál veremos en este documento?

-En el desarrollo de software, los programadores buscamos constantemente las prácticas más eficientes para construir sistemas robustos, mantenibles y escalables a lo largo del tiempo. Los desafíos cotidianos en la programación llevaron a la identificación de soluciones recurrentes, dando origen al concepto de **patrón de diseño**: una solución estandarizada, probada y reutilizable para resolver problemas comunes dentro de la programación orientada a objetos. Estos patrones no son fragmentos de código listos para copiar y pegar, sino guías conceptuales y plantillas que describen cómo estructurar clases y objetos para satisfacer necesidades específicas de manera óptima.

El propósito de este documento es profundizar en una categoría particular de estas soluciones: los **patrones estructurales**, los cuales se enfocan en cómo combinar clases y objetos para formar estructuras más amplias y flexibles. Específicamente, este trabajo práctico se centra en el patrón **Decorador** (*Decorator*), analizando cómo permite añadir nuevas funcionalidades y responsabilidades a un objeto de forma dinámica en tiempo de ejecución, y cómo se posiciona como una alternativa elegante y flexible frente a la herencia clásica.



NO COPIAR

1) El problema que resuelve:

- **El problema:**

El problema puntual que resuelve este patrón es la incapacidad de cambiar características opcionales en un objeto sin duplicar código ni crear un caos de clases, siendo más simple este patrón de diseño **hace que la herencia del objeto pasa a ser estética por ende los objetos pasan a heredar todo el bloque completo**

- **Dilema Técnico:**

Al usar herencia, la relación se define en el código antes de que el programa se ejecute (en tiempo de compilación). Obligando al programador a tomar una decisión definitiva.

Por ejemplo: tenemos un objeto el cual queremos que tenga las características: A, B, C, D y un programador quiere otro que solo tenga las características A, C, la herencia obligará a:

1-) Haber programado previamente una clase específica que se llame claseAC.

2-) Si el usuario mañana quiere un nuevo objeto que tenga las características A, B, D, se verá obligado a cambiar el código y crear la clase claseABD.

- **Los tres dolores puntuales a la hora de desarrollar:**

Problema Puntual	¿Por qué ocurre?	Ejemplo Real
Multiplicación geométrica de clases	Porque intentar cubrir todas las combinaciones matemáticas con herencia te lleva a un número absurdo de archivos de código.	Con solo 5 características opcionales, necesitarías crear más de **30 subclases** para cubrir todas las combinaciones posibles.
Código duplicado (Copy-Paste)	Porque el comportamiento de la característica **B** vas a tener que copiarlo y pegarlo dentro de la subclase `ClaseAB`, en la subclase `ClaseBC`, en la subclase `ClaseABC`, etc.	Si cambia la lógica de cómo se calcula el impuesto de la característica **B**, tenés que buscar y modificar todas las subclases combinadas donde la metiste.
Inmutabilidad de comportamiento	Porque un objeto no puede cambiar de "padre" una vez que nació.	Si tenés una interfaz de usuario donde un botón es `BotonSimple`, no podés transformarlo en un `BotonConBordeAzul` y `BotonConSombra` al pasar el mouse por encima de forma limpia, a menos que destruyas el objeto y crees uno nuevo.

NO COPIAR

- **EL problema: “Explosión de clases y Rigidez Estética**

-**caso en la vida real**:

En el desarrollo de videojuegos, uno de los desafíos mas comunes es el de gestionar las Mejoras, Estados y Habilidades Acumulativas que los personajes o ítems adquirien en tiempo de ejecución.

Imaginemos que estamos programando un videojuego de rol (RPG) y modelamos un arma básica: espada. El diseño inicial requiere que el arma inflija un daño físico base. Sin embargo, la jugabilidad exige que el usuario pueda aplicar modificadores de objeto opcionales a su arma (como Fuego, Hielo o Veneno) mediante runas, e incluso **combinar múltiples** elementos al mismo tiempo.

Código sin el patrón: El enfoque por herencia:

Intentar resolver este requerimiento mediante herencia clásica nos obliga a predefinir todas las combinaciones posibles de elementos en tiempo de compilación. El código se estructuraría de la siguiente manera;

NO COPIAR



```java
public // 1. Clase Base
public class Espada {
    public int calcularDanio() {
        return 15; // Daño físico base
    }
    public String getDescripcion() {
        return "Espada Básica";
    }
}

// 2. Subclases de efectos individuales
public class EspadaFuego extends Espada {
    @Override
    public int calcularDanio() {
        return super.calcularDanio() + 5;
    }
    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " + Fuego";
    }
}

public class EspadaHielo extends Espada {
    @Override
    public int calcularDanio() {
        return super.calcularDanio() + 3;
    }
    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " + Hielo";
    }
}

// 3. Inicio de la explosión de clases por combinación
public class EspadaFuegoYHielo extends Espada {
    @Override
    public int calcularDanio() {
        return super.calcularDanio() + 5 + 3; // Código duplicado/Hardcodeado
    }
    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " + Fuego + Hielo";
    }
}
{

}
```

¿Dónde le duele a este diseño?

Este enfoque estático genera tres problemas graves que quiebran la mantenibilidad del software:

1. Crecimiento sin control (Explosión Combinatoria)

La herencia es jerárquica y rígida. Si el juego cuenta con 4 efectos elementales y queremos permitir que el jugador los combine libremente, la matemática nos obliga a crear clases para cada permutación. Con solo agregar un elemento nuevo (por ejemplo, Electricidad), no solo debemos crear `EspadaElectrica`, sino también `EspadaFuegoYElectrica`, `EspadaHieloYElectrica`, `EspadaFuegoHieloYElectrica`, y así sucesivamente. El árbol de clases crece de forma exponencial, llenando el proyecto de decenas de archivos redundantes.

2. Código duplicado y disperso

La lógica de lo que hace cada runa individual está fragmentada. El modificador de daño de fuego (`+5`) se encuentra Hardcodeado en `EspadaFuego`, pero vuelve a repetirse en `EspadaFuegoYHielo` y en `EspadaFuegoYVeneno`. Si el día de mañana el balanceador del juego decide que el daño de fuego debe bajar a `+4`, estamos obligados a buscar y modificar manualmente todas las clases combinadas que incluyan fuego. Es un nido de bugs asegurado.

3. Mutación imposible en tiempo de ejecución

En los videojuegos, los estados cambian segundo a segundo (un hechizo de fuego dura 10 segundos, una runa se equipa o desequipa desde el inventario). Utilizando herencia, un objeto no puede cambiar su clase una vez instanciado. Si el jugador tiene una `Espada` común y activa una runa de fuego, el motor del juego se ve obligado a:

1- Destruir el objeto `Espada` actual.
2- Copiar todas sus propiedades internas (durabilidad, estadísticas).
3- Instanciar un objeto completamente nuevo de la clase `EspadaFuego`.
4- Reemplazarlo en el inventario.

Este proceso de "destruir y recrear" objetos continuamente para simular modificaciones temporales degrada el rendimiento del sistema y vuelve la lógica del inventario sumamente compleja. El patrón Decorator resuelve este dolor puntual transformando la herencia estática en una composición dinámica basada en capas.

NO COPIAR

Siendo más técnico:

Este enfoque estructural genera tres patologías arquitectónicas que degradan la mantenibilidad del software:

Patología Arquitectónica	Mecanismo del Problema	Impacto en el Sistema
Explosión Combinatoria de Clases	La herencia es unidimensional. Para soportar \(N\) comportamientos independientes y combinables, la cantidad de subclases requeridas escala a una tasa de \(2^{N}\).	El árbol de tipos crece de forma descontrolada. Agregar una sola variable nueva exige codificar subclases para todas las permutaciones existentes, saturando el espacio de nombres.
Dispersión de la Lógica y Redundancia	Al no poder encapsular una funcionalidad opcional en una única entidad aislada, la implementación de dicho comportamiento se atomiza y se copia de forma manual a través de las distintas subclases combinadas.	Violación flagrante del principio **DRY (Don't Repeat Yourself)**. Modificar el algoritmo o los parámetros de una sola característica secundaria obliga a realizar un refactor manual en múltiples archivos dispersos.
Inmutabilidad de Tipos en Tiempo de Ejecución	El tipo de un objeto queda sellado tras la instanciación. La herencia no permite alterar dinámicamente el comportamiento de una instancia según el estado del contexto de ejecución.	Para añadir o remover una funcionalidad en caliente, el sistema debe destruir el objeto actual, instanciar la subclase destino, clonar el estado interno y reasignar las referencias. Esto penaliza el rendimiento e introduce complejidad transaccional.

Este diseño viola directamente el **Principio de Abierto/Cerrado (OCP)** y el **Principio de Responsabilidad Única (SRP)**. El patrón **Decorator** resuelve este escenario transicionando de una arquitectura basada en la especialización de clases (*Is-A*) a una arquitectura basada en la **composición dinámica de interfaces (*Has-A*) por capas**.

NO COPIAR

2) ¿Por qué usar este patrón y no otro?

-Para poder justificar su uso en un ecosistema de software, no basta con entender cómo funciona; también es necesario analizar el costo-beneficio de su aplicación frente a la ausencia de patrones y frente a otras alternativas arquitectónicas de la lista del GanG of Four(Gof)

- **Por qué la solución simple deja de ser útil**:

-La solución simple ante el requerimiento de modificar un comportamiento consiste en recurrir a la **herencia lineal** o a la **adición de banderas booleanas (flags) de estado** dentro de una clase única. Ambos enfoques fallan drásticamente cuando el sistema escala bajo las siguientes condiciones:

1) **La pérdida de cohesión por acumulación de estados:** si intentamos evitar la explosión de subclases concentrando todas las características opcionales dentro de única clase masiva llena de variables condicionales (IF (TieneComportamientoA)) {…}), transformamos el componente en un **Objeto dios (God Object).** Esto rompe el principio de responsabilidad única (SRP), volviendo la clase extremadamente difícil de testear y propensa a efectos colaterales imprevistos ante cualquier cambio.

2) **La escala geométrica del mantenimiento:** La herencia clásica asume que las extensiones de funcionalidad son jerárquicas y mutuamente excluyentes (un árbol taxonómico limpio). Sin embargo, cuando las características deseadas son **ORTOGONALES** (pueden coexistir en cualquier orden o cantidad), la herencia lineal exige que el desarrollador codifique explícitamente el soporte para el futuro. La solución simple deja de servir en el momento exacto en que el costo de agregar una sola característica nueva requiere modificar o crear múltiples componentes existentes, rompiendo el **Principio de Abierto/cerrado (OCP).**

- **Conclusión Arquitectónica:**

El patrón **Decorator** emerge como la única solución óptima cuando el requerimiento técnico exige que las responsabilidades de un objeto sean **aditivas, dinámicas y transparentes** para el cliente. A diferencia de *Strategy*, que altera el "cómo" se hace la operación principal, o de *Adapter*, que altera el "cómo" se llama a la operación, el Decorator se enfoca exclusivamente en **enriquecer el "qué" hace el objeto**, preservando su identidad y su contrato intactos.

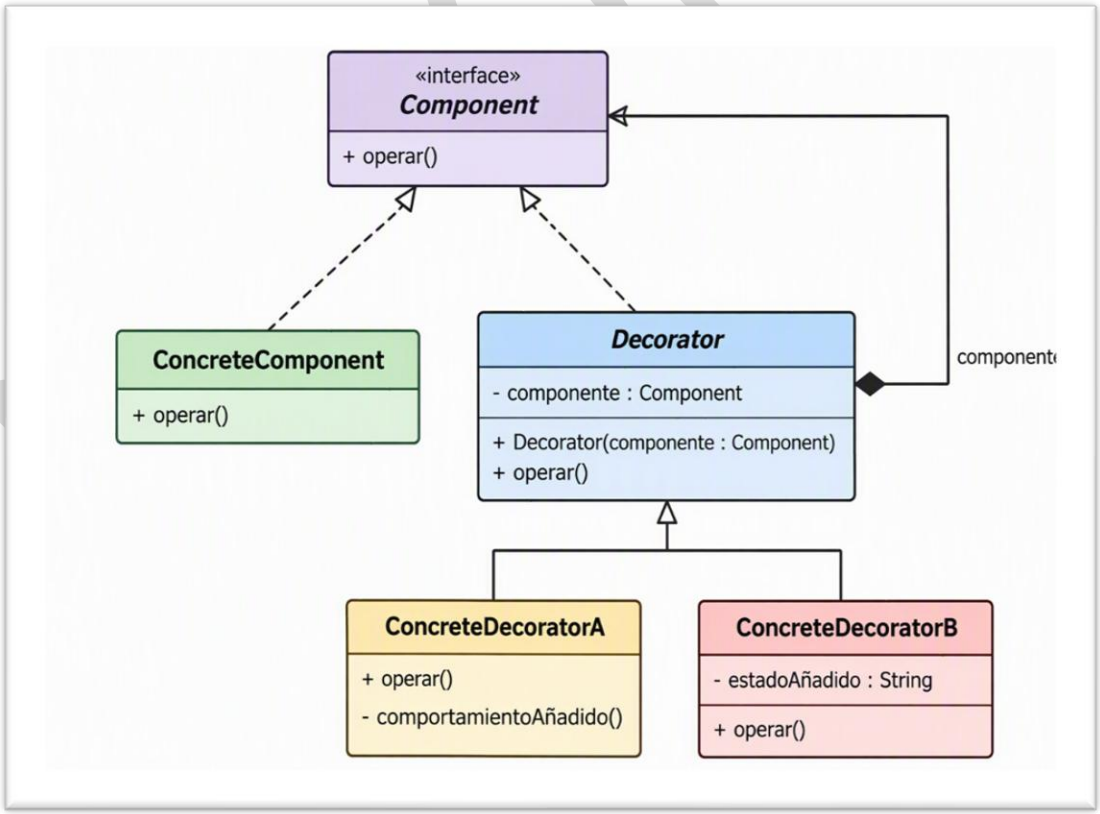
NO COPIAR

3). La Estructura del Patrón

El patrón **Decorator** basa su arquitectura en el uso de la **composición y la delegación** por encima de la herencia clásica. Esto se logra haciendo que la clase decoradora implemente la misma interfaz que el objeto que pretende envolver, manteniendo una referencia interna hacia él.

-Diagrama UML Genérico:

El siguiente diagrama de clases presenta la estructura formal y canónica del patrón decorador de acuerdo con la literatura de **Gang of Four (Gof),** utilizando abstracciones puras independientes de cualquier dominio de negocio:



- **Descripción de Roles y Responsabilidades:**

Cada componente dentro de esta estructura cumple una función específica orientada a garantizar la transparencia y la flexibilidad dinámica del sistema:

Rol Arquitectónico	Responsabilidad Técnica
Component	Define la interfaz común o el contrato abstracto para todos los objetos que pueden recibir responsabilidades adicionales de forma dinámica. Obliga a que tanto los objetos reales como sus envolturas expongan el mismo conjunto de operaciones públicas (`operar()`), garantizando la transparencia para el cliente.
ConcreteComponent	Representa al objeto básico de negocio que implementa la funcionalidad central por defecto. Define el comportamiento base sobre el cual se irán apilando las nuevas capas de responsabilidades. No tiene conocimiento de la existencia de decoradores.
Decorator	Clase abstracta que actúa como el puente de composición. Mantiene una referencia privada codificada hacia un objeto de tipo `Component` (el cual puede ser un `ConcreteComponent` u otro `Decorator`). Su implementación por defecto de `operar()` se limita a delegar la llamada directamente al componente interno enmascarado.
ConcreteDecoratorA / B	Clases concretas que extienden al `Decorator` base y materializan las responsabilidades adicionales. Sobrescriben el método `operar()` para inyectar su lógica específica (estados extras o comportamientos adicionales) de manera aditiva, ejecutando estas acciones antes, después, o en lugar de la llamada delegada al objeto interno.

NO COPIAR

- **Dinámica de Ejecución (Flujo de Mensajes):**

Cuando el cliente invoca el método `operar()` sobre el decorador más externo, se desencadena una ejecución en cadena orientada al flujo de capas:

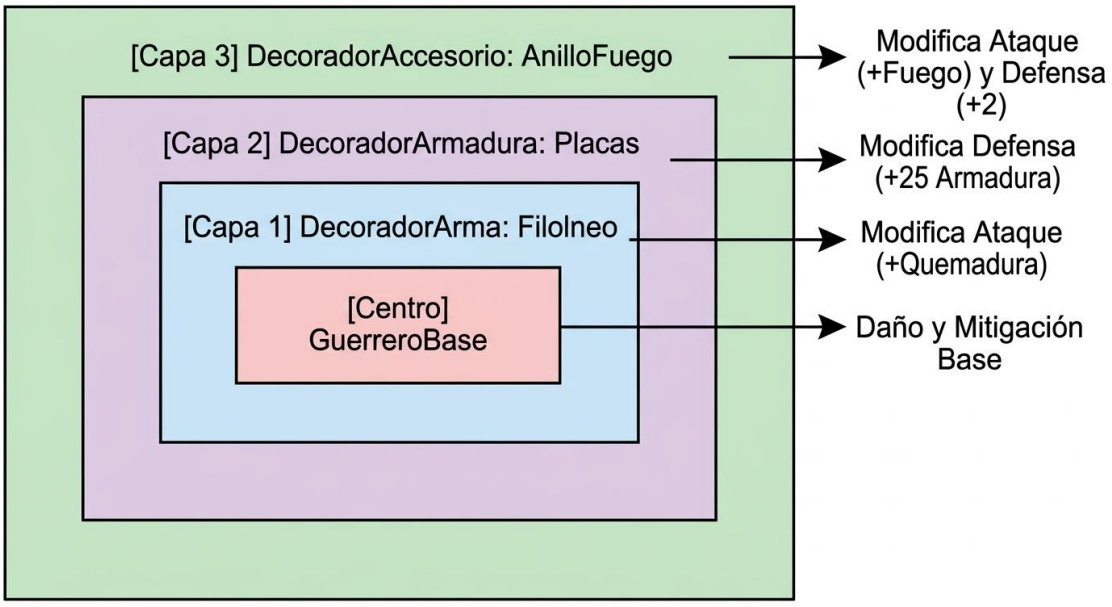
1. El decorador externo intercepta la llamada.
2. Ejecuta opcionalmente lógica previa a la delegación.
3. Invoca internamente al método `operar()` del componente que envuelve
4. (delegación).
5. El componente interno procesa su propia lógica (y repite el ciclo si es otro decorador enmascarado).
6. La llamada regresa por la pila, permitiendo al decorador externo ejecutar lógica posterior antes de retornar el control definitivo al cliente.

NO COPIAR

4). Ejemplo concreto en java

El patrón de diseño **Decorator (Decorador)** es una solución estructural que permite añadir responsabilidades y comportamientos adicionales a un objeto de forma dinámica en tiempo de ejecución. Funciona como una alternativa extremadamente flexible a la herencia clásica, permitiendo extender las capacidades de una entidad mediante la composición sin necesidad de modificar las clases existentes ni generar una explosión de subclases rígidas.

Para entender su funcionamiento de forma clara y profunda, podemos analizar el motor de combate de un videojuego de rol (RPG). Imaginemos que tenemos una entidad básica, un **GuerreroBase**, que posee estadísticas mínimas y métodos esenciales para el flujo del juego, como atacar y defenderse. En lugar de crear subclases estáticas para cada combinación posible de equipamiento, el patrón Decorador introduce el concepto de "envoltorios apilables". Cada pieza de equipo (un arma, una armadura o un accesorio) actúa como un decorador que envuelve al personaje base o a un personaje que ya ha sido envuelto previamente por otros objetos, conformando capas sucesivas similares a las de una cebolla.



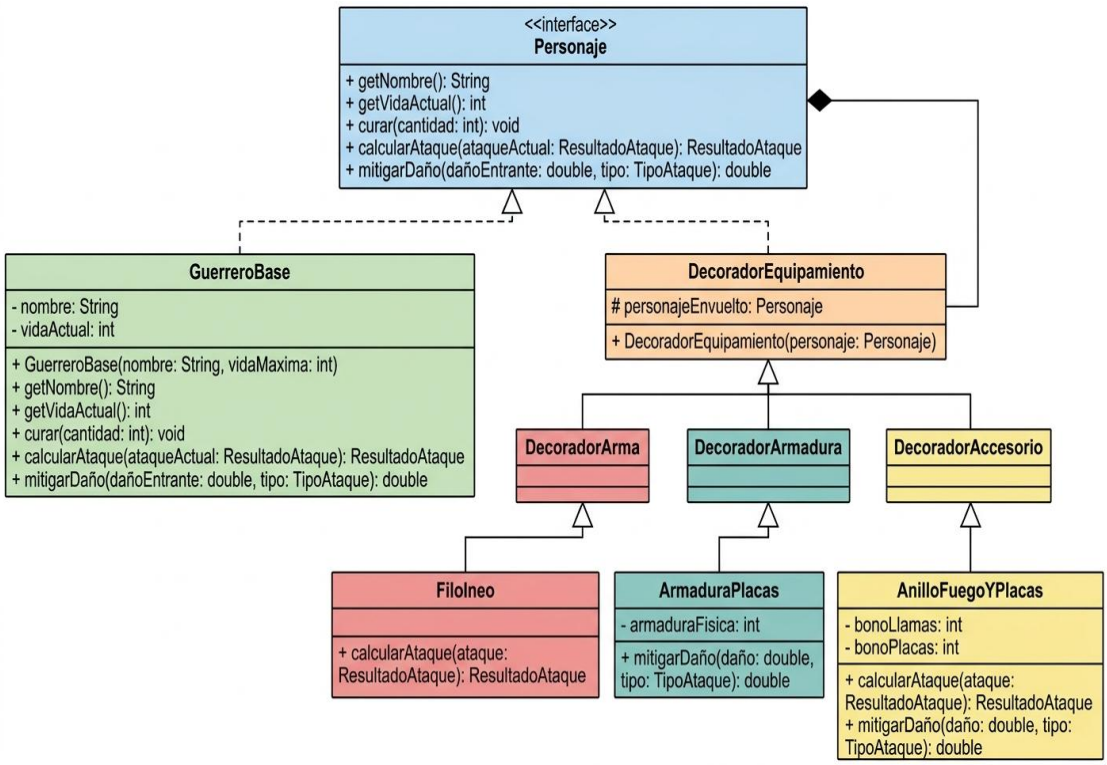
La verdadera magia del patrón en este escenario radica en la **unificación y especialización de las ranuras de equipamiento**:

- **Contrato Unificado (Personaje):** Tanto el héroe desnudo como cualquiera de sus equipamientos implementan la misma interfaz raíz. Esto expone dos flujos críticos: la ofensiva (calcularAtaque) y la defensiva (mitigarDaño). Al compartir el mismo tipo abstracto, el motor del juego puede tratar a un personaje completamente equipado exactamente de la misma manera que a uno novato.
- **Especialización por Ranuras:** El decorador base se ramifica en categorías específicas según la sección del inventario:
    - **Ranura de Arma (DecoradorArma):** Intercepta el flujo ofensivo. Por ejemplo, un *Filo Ígneo* toma el daño físico de la capa interna, le añade daño elemental de fuego y activa una bandera de estado de *Quemadura*, dejando pasar la defensa intacta hacia adentro.
    - **Ranura de Armadura (DecoradorArmadura):** Intercepta el flujo defensivo. Una *Armadura de Placas* recibe el impacto del enemigo, le resta su valor de mitigación estática y le pasa el daño reducido a la salud del personaje base.
    - **Ranura de Accesorio (DecoradorAccesorio):** Permite comportamientos híbridos avanzados. Un *Anillo de Fuego y Placas* altera ambos flujos de manera simultánea, inyectando estadísticas defensivas (+2 de armadura) y ofensivas (daño mágico de fuego) al mismo tiempo, además de poder alojar mecánicas condicionales complejas como el robo de vida del *Set de Sangre*.

Cuando el sistema invoca una acción (como realizar un ataque o recibir un golpe), la llamada **se propaga de forma recursiva a través de toda la cadena de decoradores apilados**. Cada capa añade su propia lógica de negocio o estadística y delega el resto a la capa inferior mediante composición. Esto proporciona una arquitectura completamente modular e infinitamente escalable: se pueden añadir nuevas espadas, escudos o amuletos mixtos creando clases independientes, respetando el principio de diseño de bajo acoplamiento y manteniendo el núcleo del sistema intacto.

NO COPIAR

- **Diagrama U.L.M del ejemplo**:



- Explicacion del U.M.L:

A continuación, se detalla de forma clara y directa cómo está estructurado el diagrama UML y qué función cumple cada uno de sus componentes dentro de la arquitectura del software.

📘 1. El Contrato Unificado (<<interface>> Personaje)

Ubicada en la parte superior (color azul), esta interfaz define el comportamiento obligatorio de cualquier entidad o equipamiento en el juego. Su propósito técnico es **garantizar el polimorfismo**. Expone dos flujos críticos:

- **Ofensivo (calcularAtaque):** Recibe un paquete con los datos del golpe y le aplica modificaciones.
- **Defensivo (mitigarDaño):** Recibe el impacto del enemigo y calcula la reducción antes de restar salud.

🟢 2. El Núcleo del Sistema (GuerreroBase)

Ubicado a la izquierda (color verde), representa al personaje real y "desnudo". Contiene los atributos de estado físicos (nombre y vidaActual). Es la clase encargada de definir el comportamiento por defecto: genera el daño físico básico y resta la vida de manera definitiva cuando el daño supera sus defensas.

🟠 3. El Envoltorio Base (DecoradorEquipamiento)

Ubicado a la derecha (color naranja), es la clase abstracta clave del patrón. Su característica principal es que posee una **relación de composición consigo misma** (representada por la línea con el rombo negro que vuelve hacia la interfaz). Esto significa que guarda internamente una referencia al # personajeEnvuelto. Su única tarea es delegar las llamadas de ataque y defensa hacia la capa inmediatamente inferior de forma transparente.

🟥🟪🟧 4. Las Ranuras de Inventario Especializadas

En el nivel intermedio inferior, el decorador se divide en tres ramas abstractas para emular el inventario del RPG:

- **DecoradorArma (Rojo):** Especializado en interceptar y potenciar las mecánicas de ataque.
- **DecoradorArmadura (Celeste):** Especializado en interceptar y mitigar el daño entrante.
- **DecoradorAccesorio (Amarillo):** Diseñado para albergar efectos de utilidad o combinados.

⚔️🛡️💍 5. Las Capas Concretas (Equipamiento Real)

Ubicadas en la base del diagrama, son las piezas de equipo que el jugador se coloca en tiempo de ejecución:

- **FiloIneo:** Sobreescribe solo el método de ataque para inyectar fuego y activar la quemadura.
- **ArmaduraPlacas:** Posee el atributo - armaduraFisica y sobreescribe solo el método de mitigación para reducir los golpes.
- **AnilloFuegoYPlacas:** Es el **híbrido**. Al ser un accesorio mágico, posee atributos para ambas estadísticas y sobreescribe **tanto el ataque como la defensa simultáneamente**, alterando los dos flujos a la vez al recorrer la cadena.

NO COPIAR

- **Código de ejemplo en java : (personaje RPG)**

1. Estructuras de Soporte: Tipos de Daño y Reportes

En un motor de RPG real, las estadísticas no pueden ser simples números enteros aislados si queremos efectos cruzados. Necesitamos objetos que encapsulen y transporten toda la información del combate a través de la cadena de envolturas.

Explicación de los Componentes

- **TipoAtaque (Enumeración):** Define las firmas elementales que soporta el sistema. Permite que las armaduras identifiquen mediante condicionales si el daño entrante es físico, mágico o de estado alterado.
- **ResultadoAtaque (Clase de Dominio):** Es una estructura de datos mutable que se pasa como parámetro y retorno en el flujo ofensivo. Al nacer, solo lleva daño físico, pero a medida que viaja por las distintas capas de equipamiento (Armas o Anillos), cada decorador altera sus variables internas de forma acumulativa y enciende las banderas de estado correspondientes.

NO COPIAR

TipoAtaque.java

```java
// TipoAtaque.java
public enum TipoAtaque {
    FISICO,
    FUEGO,
    SANGRE
}
```

ResultadoAtaque.java

```java
// ResultadoAtaque.java
public class ResultadoAtaque {
    public int dañoFisico;
    public int dañoFuego;
    public int dañoSangre;
    public boolean aplicaQuemadura;

    // Constructor inicial: un golpe siempre empieza con daño físico base
    public ResultadoAtaque(int dañoFisico) {
        this.dañoFisico = dañoFisico;
        this.dañoFuego = 0;
        this.dañoSangre = 0;
        this.aplicaQuemadura = false;
    }

    // Calcula de forma dinámica el impacto total sumando todos los elementos
    public int getDañoTotal() {
        return dañoFisico + dañoFuego + dañoSangre;
    }
}
```

2. El Contrato Raíz: La Interfaz Personaje

Este es el pilar estructural del patrón Decorador. Define la firma de los métodos que unifican el comportamiento tanto del héroe como de cualquier ítem equipado en su inventario.

Explicación del Componente

La interfaz unifica los dos flujos vitales del ciclo de combate:

1. **Flujo Ofensivo (calcularAtaque):** Recibe un objeto ResultadoAtaque, le aplica modificaciones geométricas o aritméticas y lo retorna hacia la siguiente capa.
2. **Flujo Defensivo (mitigarDaño):** Recibe el impacto original del atacante y el elemento. Devuelve el número neto final que impactará en los puntos de salud.

Personaje.java

```java
// Personaje.java
public interface Personaje {
    String getNombre();
    int getVidaActual();
    void curar(int cantidad);

    // Flujo Ofensivo: muta el paquete de daño entrante
    ResultadoAtaque calcularAtaque(ResultadoAtaque ataqueActual);

    // Flujo Defensivo: reduce el daño entrante antes de restar vida
    double mitigarDaño(double dañoEntrante, TipoAtaque tipo);
}
```

NO COPIAR

3. El Componente Concreto: GuerreroBase

Representa la entidad básica sobre la cual se van a ir acoplando las piezas del inventario. Es el final de la cadena de llamadas recursivas.

Explicación del Componente

- **Gestión de Estado:** Almacena los atributos físicos de salud (vidaActual) y la identidad del héroe (nombre).
- **Comportamiento Base:** Implementa las respuestas por defecto del juego. Cuando ataca, genera un reporte limpio que solo contiene daño físico plano. Cuando defiende, reduce el daño usando una resistencia básica de la piel contra golpes físicos, pero deja pasar intactos los elementos mágicos.

GuerreroBase.java

```java
// GuerreroBase.java
public class GuerreroBase implements Personaje {
    private String nombre;
    private int vidaActual;

    public GuerreroBase(String nombre, int vidaMaxima) {
        this.nombre = nombre;
        this.vidaActual = vidaMaxima;
    }

    @Override public String getNombre() { return this.nombre; }
    @Override public int getVidaActual() { return this.vidaActual; }

    @Override
    public void curar(int cantidad) {
        this.vidaActual += cantidad;
        System.out.println("-> [SISTEMA] " + nombre + " se cura " + cantidad + " HP. Vida actual: " + vidaActual);
    }

    @Override
    public ResultadoAtaque calcularAtaque(ResultadoAtaque ataqueActual) {
        // Fin de la cadena ofensiva: retorna el paquete físico inicial sin mutaciones elementales
        return ataqueActual;
    }

    @Override
    public double mitigarDaño(double dañoEntrante, TipoAtaque tipo) {
        // Fin de la cadena defensiva: mitigación por defecto del personaje desnudo
        if (tipo == TipoAtaque.FISICO) {
            return Math.max(0, dañoEntrante - 5); // Reducción mínima por armadura natural
        }
        return dañoEntrante; // El daño elemental pasa directo si no hay equipo mágico
    }
}
```

4. El Decorador Base: DecoradorEquipamiento

Esta es la superclase abstracta intermedia de la que nacerán todas las categorías de equipamiento. Es el pegamento estructural del patrón.

Explicación del Componente

- **Composición en Acción:** Su atributo clave es # personajeEnvuelto, el cual apunta a la interfaz raíz Personaje. Gracias a esto, la clase puede albergar en su interior tanto a un objeto concreto (GuerreroBase) como a otro decorador ya anidado.
- **Transparencia de Delegación:** Implementa todos los métodos de la interfaz delegando directamente en el objeto envuelto (personajeEnvuelto.metodo()). Si una subclase decide no sobreescribir un flujo (por ejemplo, una espada que no otorga defensa), el sistema se comporta de manera transparente y deja pasar la llamada sin alterar los datos.

DecoradorEquipamiento.java

```java
// DecoradorEquipamiento.java
public abstract class DecoradorEquipamiento implements Personaje {
    protected Personaje personajeEnvuelto;

    public DecoradorEquipamiento(Personaje personaje) {
        this.personajeEnvuelto = personaje;
    }

    @Override public String getNombre() { return personajeEnvuelto.getNombre(); }
    @Override public int getVidaActual() { return personajeEnvuelto.getVidaActual(); }
    @Override public void curar(int cantidad) { personajeEnvuelto.curar(cantidad); }

    @Override
    public ResultadoAtaque calcularAtaque(ResultadoAtaque ataqueActual) {
        // Delega la llamada de ataque hacia la capa interior
        return personajeEnvuelto.calcularAtaque(ataqueActual);
    }

    @Override
    public double mitigarDaño(double dañoEntrante, TipoAtaque tipo) {
        // Delega la llamada de defensa hacia la capa interior
        return personajeEnvuelto.mitigarDaño(dañoEntrante, tipo);
    }
}
```

5. <u>Las Capas de Ranuras: Arma, Armadura y Accesorio</u>

Estas tres clases abstractas extienden de DecoradorEquipamiento. Su propósito técnico es segmentar la jerarquía del patrón para organizar el inventario del juego y permitir la adición de comportamientos específicos por categoría en el futuro.

DecoradorArma.java

```java
// DecoradorArma.java
public abstract class DecoradorArma extends DecoradorEquipamiento {
    public DecoradorArma(Personaje personaje) {
        super(personaje);
    }
}
```

DecoradorArmadura.java

```java
// DecoradorArmadura.java
public abstract class DecoradorArmadura extends DecoradorEquipamiento {
    public DecoradorArmadura(Personaje personaje) {
        super(personaje);
    }
}
```

DecoradorAccesorio.java

```java
// DecoradorAccesorio.java
public abstract class DecoradorAccesorio extends DecoradorEquipamiento {
    public DecoradorAccesorio(Personaje personaje) {
        super(personaje);
    }
}
```

6. Implementaciones Concretas de Equipamiento

Estas son las clases reales que el usuario instanciará para envolver al personaje. Cada una intercepta un flujo específico (u ambos) para inyectar su lógica de negocio.

Explicación de los Componentes

- **FiloIneo (Ranura Arma):** Sobreescribe exclusivamente el flujo ofensivo. Toma el paquete de ataque, incrementa el daño elemental de fuego y enciende la bandera de estado de quemadura. Deja pasar la defensa intacta a través de super.mitigarDaño.
- **ArmaduraPlacas (Ranura Armadura):** Sobreescribe exclusivamente el flujo defensivo. Intercepta el daño entrante y, si es de tipo físico, le resta su valor estático (armaduraFisica) antes de propagarlo al interior. Deja pasar el ataque intacto.
- **AnilloFuegoYPlacas (Ranura Accesorio - Híbrido):** Al ser un objeto místico, intercepta **ambos flujos simultáneamente**. Añade daño de fuego en la ofensiva y resta daño plano en la defensiva, demostrando la flexibilidad absoluta del contrato unificado.

FiloIneo.java

```java
// FiloIneo.java
public class FiloIneo extends DecoradorArma {
    public FiloIneo(Personaje personaje) {
        super(personaje);
    }

    @Override
    public ResultadoAtaque calcularAtaque(ResultadoAtaque ataqueActual) {
        // 1. Propaga la llamada hacia las capas más internas primero
        ResultadoAtaque ataque = super.calcularAtaque(ataqueActual);

        // 2. Inyecta su propio comportamiento ofensivo elemental
        ataque.dañoFuego += 35;
        ataque.aplicaQuemadura = true;
        System.out.println("[Efecto Filo Ígneo]: Las llamas envuelven el arma (+35 Daño Fuego, Aplica Quemadura).");

        return ataque;
    }
}
```

ArmaduraPlacas.java

```java
// ArmaduraPlacas.java
public class ArmaduraPlacas extends DecoradorArmadura {
    private int armaduraFisica;

    public ArmaduraPlacas(Personaje personaje) {
        super(personaje);
        this.armaduraFisica = 25; // Aporte de protección estática
    }

    @Override
    public double mitigarDaño(double dañoEntrante, TipoAtaque tipo) {
        // Intercepta solo si el impacto enemigo es de tipo físico
        if (tipo == TipoAtaque.FISICO) {
            double dañoReducido = Math.max(0, dañoEntrante - this.armaduraFisica);
            System.out.println("[Armadura de Placas]: Absorbiendo " + this.armaduraFisica + " de daño físico.");
            // Propaga el daño ya reducido hacia la capa interior
            return super.mitigarDaño(dañoReducido, tipo);
        }

        // Si es daño elemental o de estado, pasa directo a la siguiente capa
        return super.mitigarDaño(dañoEntrante, tipo);
    }
}
```

AnilloFuegoYPlacas.java

```java
// AnilloFuegoYPlacas.java
public class AnilloFuegoYPlacas extends DecoradorAccesorio {
    private int bonoLlamas;
    private int bonoPlacas;

    public AnilloFuegoYPlacas(Personaje personaje) {
        super(personaje);
        this.bonoLlamas = 15;
        this.bonoPlacas = 7;
    }

    @Override
    public ResultadoAtaque calcularAtaque(ResultadoAtaque ataqueActual) {
        ResultadoAtaque ataque = super.calcularAtaque(ataqueActual);
        ataque.dañoFuego += this.bonoLlamas;
        System.out.println("[Anillo Mistico]: El brillo rúnico añade +" + this.bonoLlamas + " de Daño de Fuego.");
        return ataque;
    }

    @Override
    public double mitigarDaño(double dañoEntrante, TipoAtaque tipo) {
        if (tipo == TipoAtaque.FISICO) {
            double dañoReducido = Math.max(0, dañoEntrante - this.bonoPlacas);
            System.out.println("[Anillo Mistico]: Desviando pasivamente +" + this.bonoPlacas + " de daño fisico.");
            return super.mitigarDaño(dañoReducido, tipo);
        }
        return super.mitigarDaño(dañoEntrante, tipo);
    }
}

```

7. Orquestación del Sistema: Main (Game Loop)

Esta clase actúa como el cliente de nuestro patrón de diseño. Su propósito es instanciar al personaje básico y acoplarle dinámicamente las distintas ranuras de equipamiento para luego testear los flujos de ataque y mitigación contra amenazas del entorno.

Explicación del Componente

- **Envoltura Dinámica: (Líneas 9 a 13):** Notá cómo la variable heroe (que es de tipo abstracto Personaje) es reasignada de forma sucesiva, envolviéndose a sí misma en nuevas instancias de equipamiento (FiloIneo, ArmaduraPlacas y el híbrido AnilloFuegoYPlacas). Al finalizar este bloque, el objeto real en memoria tiene la estructura exacta de la infografía de capas concéntricas ("cebolla") que definimos previamente.
- **Polimorfismo en Acción:** Cuando el motor invoca los métodos .calcularAtaque() o .mitigarDaño(), el bucle principal no sabe (ni necesita saber) qué piezas tiene puestas el personaje. La llamada golpea la capa más externa, y la arquitectura del patrón Decorador se encarga de propagar y acumular los efectos de manera automática.

Main.java

```java
// Main.java
public class Main {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("--- CONFIGURACIÓN DEL PERSONAJE ---");

        // 1. Instanciamos nuestro componente concreto (Héroe Base)
        Personaje heroe = new GuerreroBase(nombre: "Arthas", vidaMaxima: 500);
        System.out.println("Se ha creado a " + heroe.getNombre() + " con " + heroe.getVidaActual() + " HP.");

        // 2. Equipamos las ranuras de inventario de forma dinámica mediante envolturas
        heroe = new FiloIneo(heroe);            // Ranura Arma
        heroe = new ArmaduraPlacas(heroe);      // Ranura Armadura
        heroe = new AnilloFuegoYPlacas(heroe);  // Ranura Accesorio (Híbrido)

        System.out.println("\n--- SIMULACIÓN DE FLUJO OFENSIVO (CALCULAR ATAQUE) ---");
        // Inicializamos un paquete de ataque físico base con 50 puntos
        ResultadoAtaque reporteEnvio = new ResultadoAtaque(dañoFisico: 50);

        // Ejecutamos la llamada polimórfica que recorrerá todas las capas
        ResultadoAtaque golpeFinal = heroe.calcularAtaque(reporteEnvio);

        // Imprimimos el reporte final acumulado consolidado por el patrón
        System.out.println("\n[REPORTE DE DAÑO EMITIDO]:");
        System.out.println("-> Daño Físico Neto: " + golpeFinal.daÑoFisico);
        System.out.println("-> Daño Fuego Elemental: " + golpeFinal.daÑoFuego);
        System.out.println("-> Daño Total Combinado: " + golpeFinal.getDaÑoTotal());
        System.out.println("-> ¿Aplica Estado Alterado (Quemadura)?: " + golpeFinal.aplicaQuemadura);

        System.out.println("\n--- SIMULACIÓN DE FLUJO DEFENSIVO (RECIBIR IMPACTO) ---");
        // Simulamos que una criatura enemiga nos asesta un golpe físico de 100 puntos
        double daÑoImpactoEnemigo = 100.0;
        System.out.println("¡Alerta! Un monstruo te ataca con " + daÑoImpactoEnemigo + " de Daño Físico.");

        // La llamada ingresa por la capa externa (Anillo) -> Placas -> GuerreroBase
        double daÑoNetoSufriendo = heroe.mitigarDaño(daÑoImpactoEnemigo, TipoAtaque.FISICO);

        System.out.println("\n[REPORTE DE MITIGACIÓN]:");
        System.out.println("-> Daño que superó las capas y golpeó la salud: " + daÑoNetoSufriendo + " HP.");
    }
}
```

NO CO

5). Uso Real y Cuándo NO Usarlo

El patrón **Decorator** es una herramienta de diseño de gran valor, pero su aplicación indiscriminada introduce una complejidad técnica innecesaria. Al igual que cualquier patrón de diseño, tiene un costo de abstracción que solo debe pagarse si las necesidades del sistema lo justifican.

- **Un caso concreto donde el patrón sobra o empeora el diseño:**

Imaginemos el desarrollo de un módulo de **Pasarela de Pagos** para una plataforma de comercio electrónico. El sistema interactúa con un procesador externo (por ejemplo, Stripe o PayPal). Inicialmente, la pasarela solo requiere ejecutar un cargo básico mediante el método `procesarPago(double monto)`.

El equipo de desarrollo anticipa que en el futuro se podrían requerir "añadidos" u operaciones adicionales sobre el flujo del pago, como:

1. **Registrar auditoría** en la base de datos local antes del cobro.
2. **Enviar una notificación** por correo electrónico al cliente si el pago es exitoso.
3. **Calcular y retener un impuesto** impositivo local según el país.

NO COPIAR

El error: Implementar Decorator de forma prematura

Un arquitecto sobre-entusiasta decide implementar el patrón Decorator. Crea una interfaz `ProcesadorPago`, un `ComponenteConcretoStripe`, y tres decoradores: `AuditoriaPagoDecorator`, `NotificacionPagoDecorator` e `ImpuestoPagoDecorator`.

- Este diseño **empeora drásticamente el sistema** por las siguientes razones técnicas:

Factor de Falla	Por qué el Decorator destruye el diseño en este caso
Incompatibilidad de Interfaz	El decorador exige mantener la misma firma exacta del método (`procesarPago(monto)`). Sin embargo, el `ImpuestoPagoDecorator` necesita saber el código postal del usuario, y el `NotificacionPagoDecorator` requiere el email del cliente. Forzar estas variables dentro de una interfaz genérica obliga a pasar un objeto de contexto masivo o corromper el contrato base.
Pérdida de Control Transaccional	Los pagos requieren atomicidad estricta (si la auditoría falla o el cobro se cae, se debe hacer un *rollback* preciso). Al delegar ciegamente en una pila de objetos envueltos en tiempo de ejecución, el manejo de excepciones y la gestión de la transacción (saber exactamente en qué capa falló y revertir los efectos secundarios) se vuelve un laberinto de código propenso a bugs financieros.
Dependencia de Orden Crítica	En el procesamiento de streams (como el Punto 4), el orden de las capas puede variar. Pero en una pasarela de pagos, el orden es mandatorio: primero se calcula el impuesto, luego se cobra, y al final se audita y notifica. Si el orden de las dependencias es rígido y obligatorio, **Decorator pierde su propósito de combinación dinámica** y se convierte en un mecanismo frágil; si el cliente invoca el pipeline en un orden incorrecto, el sistema rompe las reglas de negocio.

NO COPIAR

- # La solución simplificada (Cómo debió resolverse)

Este escenario no requería una estructura de envolturas dinámicas por capas. La solución más limpia, mantenible y legible consistía en aplicar un enfoque procedimental simple o delegar las tareas secundarias a través de un patrón **Observer** (Eventos) o una inyección de dependencias lineal en un servicio coordinador de pagos:

- **Regla de oro para el diseño:**

  - Conviene evitar el patrón Decorator si:

  - El comportamiento de las extensiones no es verdaderamente ortogonal ni combinable de forma aleatoria.
  - El orden de ejecución de los componentes adicionales es estrictamente fijo.
  - Las extensiones de comportamiento requieren parámetros específicos que alteran o ensucian la interfaz común del componente original.

6). Fuentes:

- Wikipedia.org: https://es.wikipedia.org/wiki/Decorator_(patr%C3%B3n_de_dise%C3%B1o)

- Refactoring.guru:

https://refactoring.guru/es/design-patterns/decorator

- Youtube : Canales y videos:

- DECORATOR | Patrones de diseño de BettaTech
- Patrón de Diseño DECORATOR | Explicación de Manuel Zapata
- Decorator explido fácil | Patron de diseño con ejemplos reales

NO COPIAR